

or even across multiple virtual machines. This allows the deployment of different nodes as soon as the resources become available or even when more nodes are needed. It allows a Platform-as-a-Service (PaaS) style of deployment and the scaling for various systems like Apache Cassandra, Riak or MongoDB. Docker also helps in simplifying the creation and operation of task queues and other such distributed systems. And, most importantly, Docker is open source, which means that anyone can contribute to it and even extend it to meet their own needs in case they need additional features that aren't available.

Differences between Docker and containers

1. There is some confusion when we try to compare containers and Docker. The latter has become quite synonymous with container technology because it has been the most successful at popularising this technology. Docker is an open source software containerisation platform, which helps in creating, deploying, and running various applications by using containers. But, we need to be aware that container technology is not new. It was built into Linux in the form of LXC almost 10 years ago, and similar virtualisation at operating system level has been offered by AIX Workload Partitions, FreeBSD jails and Solaris containers as well.
2. Today, Docker is not the only tool for Linux which helps in software containerisation. There are other alternatives like Rkt also available in the market. Rkt is a command line tool, which is used for running different app containers produced by CoreOS. It is able to handle both Docker containers as well as the ones that comply with its App Container Image specification.
3. One of the most important reasons for launching Rkt is that Docker has become too big and has actually lost its simplicity. Containers help in reliably running any type of software, whereas Docker is now building tools to launch various systems for clustering, cloud servers, and a wide range of other functions like building and running images, uploading or downloading them and eventually even overlay networking—all compiled into a single, monolithic binary running primarily as the root on any server.
4. According to Kelsey Hightower, chief advocate of CoreOS, other container images are intended to be much more secure than Docker images because they are signed by their creators. When we use Docker and pull an App Container image, we can decide if we trust the developer before running it. Rkt can also run Docker images, but they won't always be signed.

Integration of Docker with several other tools

Since Docker provides a platform to deploy an application with its complete package, it's quite obvious that it should have the ability to integrate with other kinds of applications as

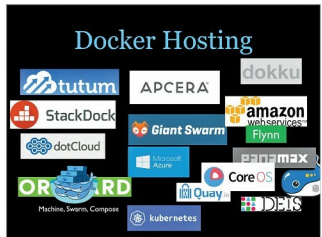


Figure 4: Various tools on which Docker can be hosted (Image credits: Google images)

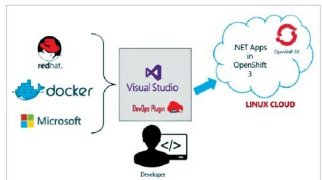


Figure 5: Role of Docker in Visual Studio 2015 (Image credits: Google images)

well, so that the application deployed on it can interact with it, without any blockages. There are several infrastructure tools like Amazon Web Services, Ansible, Chef, CFEngine or Google Cloud platform, which can easily be integrated with Docker.

Docker can also interact with other tools like Jenkins, IBM Bluemix, Microsoft Azure, OpenStack Nova, Salt, Vagrant and VMware vSphere Integrated Containers. The Cloud Foundry Diego integrates Docker into the Cloud Foundry PaaS. The OpenShift PaaS of Red Hat integrates Docker and other related projects (Kubemetes, Project Atomic, Gear and others) since v3 (June 2015). The Apprenda PaaS integrates containers of Docker in version 6.0 of its product.

Stack has complete support for automatically performing builds inside Docker, using the user ID and volume mounts switching to make it mostly seamless. FP Complete provides images for use with Stack that also include other tools, GHC and optionally, have all of the Stackage LTS packages which are pre-installed in the global package database. The main purpose for using Stack/Docker this way is to ensure that all developers build in a consistent environment without any of the team members needing to deal with Docker on their own.